

COS 214 Practical 2 Submission

Hayley Nel u25101821

Matthew Malan u25009801

Task 1

1.1

Subsystem	Trying to be	Should be
Movement	Strategy	State
Routes	State	Strategy
Map	Composite	Composite
Place Features	Decorator	Decorator
Biomes	Abstract Factory	Abstract Factory

The pairs that are swapped are movement and routes. The intent of the Strategy design pattern is to make interchangeable, encapsulated algorithms which vary independently from the clients that use it. This allows the system or user to choose which algorithm to use to solve a problem. In our case, this would be ideal then to choose which route plotting algorithm to use depending on priority (shortest, fastest, scenic, cheapest etc). And so Strategy should be used rather than State. The intent of the State design pattern is to allow an object to change its behaviour when its internal state changes. This is optimal for movement because when Traveller's internal state (modeCode) changes, which the object's behaviour is dependent on, then execution will be redirected to the appropriate method needed (doMove()) in the concrete state.

1.2

Subsystem	Design Pattern	Participants
Movement	State	State – TravelMode ConcreteState – WalkingMode, CycleMode, ZiplineMode, SwimMode, BoatMode Context - Traveller
Routes	Strategy	Strategy – RouteStrategy ConcreteStrategy – ShortestRoute, FastestRoute, ScenicRoute Context - Trip
Map	Composite	Component - Map Leaf – Location Composite – Region Client – main.cpp
Place Features	Decorator	Component –Map

		ConcreteComponent – Location Decorator –PlaceDecorator ConcreteDecorators – questDecorator, questBoardDecorator
Biomes	Abstract Factory	AbstractFactory – BiomeFactory ConcreteFactory – DungeonFactory, DesertFactory, OceanFactory AbstractProductA – Terrain AbstractProductB – NPC AbstractProductC – Obstacle ConcreteProduct – DesertNPC, DungeonNPC, OceanNPC, OceanTerrain, DungeonTerrain etc. Client -main.cpp

1.3

Wrong pattern choice:

- RouteState contains a { if / else chain } in getRoute(), not concrete states
Rule: If using the state pattern, the behaviour should change using changes in state, not an if / else chain. aThese implementations should be in separate concrete classes.
Consequence: If you were to extend this, you would have to edit that chain. This violates open/close principle
- MoveStrategy contains a big switch
Rule: Strategy pattern should have different algorithms in different concrete strategy classes
Consequence: You cannot easily change between algorithms at runtime and the purpose of strategy is defeated

Wrong UML notation:

- The decorator relationships use inheritance arrows from the combined decorators to Location
Rule: Decorators should have a composition/association to the wrapped component, not inherit from it.
Consequence: The program will not work if Location is inheriting from it's concrete classes
- Abstract Factory is incomplete
Rule: AF uses concrete factory classes and concrete products. The UML only has the concrete products

Consequence: Nothing implements the abstract operations to produce the concrete product objects

Broken Ownership:

- Map appears aggregates Location/Region objects rather than owning them

Rule: Since map owns the objects in it's vectors, then there should be composition

Consequence: Leaks, incorrect deletion, dangling pointers

Missing virtual destructor:

- base classes such as Location have no virtual destructor shown

Rule: Polymorphic deletion requires a virtual destructor to delete a derived object through a base pointer.

Consequence: Using a non-virtual base destructor would leave the derived part undestroyed.

Excessive coupling:

- GameManager directly knows about Traveller, MoveStrategy, RouteState, Map, and WorldBuilder

Rule: Coordination should depend on abstractions and delegate responsibilities to the objects that own them

Consequence: GameManager becomes difficult to change because changes to subsystems affect it

- GameManager directly depends on the concrete products of the factory

Rule: purpose of Abstract Factory is to hide concrete product creation behind an abstract factory interface

Consequence: Adding a new biome or product forces changes to code that should not need to know about concrete products.

1.4

GameManager has been made to know about everything. It is connected to every part of the design and indirectly to concrete classes. Centralising all behaviour in one class defeats the coupling goals the patterns exist to serve because they are supposed to move responsibilities away from a central controller so that a change in a subsystem doesn't require a change to GameManager.

What should be done instead:

GameManager should coordinate only the major objects and the objects responsible for a concept should own that concept's behaviour.

Movement behaviour should belong to different state objects. Traveller should own it's current TravelMode which asks the concrete classes to perform movement based on the state specifics

Route behaviour should belong to different strategy objects. Trip should have a Route Strategy to ask to calculate the route

Map management should belong to decorator objects. Map should own it's location and Region components and be responsible for their lifetimes.

Location behaviour should belong to decorator objects. Decorator objects should own/wrap another Location.

Object creation should belong to factories

1.5

It is difficult to read because:

There are many crossing/ diagonal relationships from GameManager down through the diagram.

The classes are spread in a way making it difficult to read. Some to the left, others to the right and more to the bottom

The 5 patterns aren't clearly separated into logical sections

An improvement would be to arrange each pattern into its own clear region but on the same level below GameManager at the top and then work downwards with their subclasses. So inheritance arrows are vertical and should not cross through unrelated classes

Task 2

THE UML IS COMING I PROMISE!!!

2.2

Abstract Factory, NPCs and Biomes

Abstract factory is used as an interface to create families of objects. In this case, we separate families by biome, e.g. Desert, Ocean. Each factory is used to construct three different products Terrain, Obstacle and NPC. We use Abstract Factory to provide an interface to the concrete factories. This ensures consistency in inheritance and allows us to make use of polymorphism when utilising factories.

BiomeFactory: the *Abstract factory*, an interface which provides basic construction operations. createNPC(), createTerrain(), createObstacle().

ConcreteFactories: Inheriting from Biomefactory, these classes implement the the virtual functions declared in BiomeFactory.

NPC: The first *Abstract Product Class*, an interface that provides the concrete classes with basic interactions to have with NPCs, such as any dialogue or information they might share.

ConcreteNPCs: Inheriting from the NPC interface class, these classes implement the virtual functions provided by NPC to make each biome have a unique interaction.

Terrain: The second *Abstract Product Class*, an interface that provides the concrete terrain classes with basic functionality and information about the biome's environment, such as descriptions or terrain-specific properties.

ConcreteTerrains: Inheriting from the Terrain interface class, these classes implement the virtual functions provided by Terrain to give each biome its own unique environment and characteristics.

Obstacle: The third *Abstract Product Class*, an interface that provides the concrete obstacle classes with basic functionality for obstacles that the Traveller may encounter during their journey. This allows obstacles to interact with the Traveller while remaining independent of the specific biome.

ConcreteObstacles: Inheriting from the Obstacle interface class, these classes implement the virtual functions provided by Obstacle to create obstacles specific to each biome.

Composite, Decorator and Map implementation

The Composite pattern allows individual objects and groups of objects to be treated in the same way. In this case, both Location and Region inherit from Map, meaning the program can add either one to a Region and call describe() without needing to know whether it is dealing with a single location or a collection of locations.

The Decorator pattern allows these additional features to be combined dynamically. A decorator can wrap another decorator, meaning multiple behaviours can be applied to the same map element. New features are able to be added to decorator seamlessly, as it only relies on inheritance of LocationDecorator.

Map: The *Component interface* used by both the Composite and Decorator patterns. It allows individual locations, regions, and decorators to be treated uniformly through polymorphism.

Location: The *Leaf component* of the Composite pattern. A Location represents an individual place in the world and provides its own description.

Region: The *Composite component* of the Composite pattern. A Region can contain multiple Map objects, including other Regions and Locations. Thus a Region takes ownership of those objects and controls their lifecycle. When the Root goes out of scope so does the rest of the tree.

PlaceDecorator: The *Decorator base class* used to dynamically add functionality to existing Map objects.

Concrete Decorators: Inheriting from PlaceDecorator, these classes add additional features or behaviour to existing locations or regions without modifying the original classes.

State and Traveller

The State pattern allows the Traveller's behaviour to change when its current TravelMode changes. The behaviour can be switched at runtime without modifying the Traveller class.

Traveller: The *Context* class in the State pattern. The Traveller maintains a reference to its current TravelMode and delegates travel-related behaviour to that current state. This allows the Traveller's movement behaviour to change depending on its current travel mode without changing the Traveller class itself.

TravelMode: The *State interface* of the State pattern. It defines the common move() operation that all concrete travel modes must implement. The Traveller can therefore interact with different travel modes through the same interface.

Concrete Travel Modes: These classes inherit from TravelMode and implement the move() function with behaviour specific to each travel mode. Each concrete state determines how the Traveller moves and can use the Traveller's current information.

Strategy and Route Strategy

The Strategy pattern allows the route-planning algorithm to be changed independently of the Trip class. Instead of placing all possible route algorithms inside Trip, the algorithms are separated into individual strategy classes, which can easily be added as they all inherit from the same class. Trip delegates the route-planning operation to its currently selected strategy.

RouteStrategy: The *Strategy interface* of the Strategy pattern. It defines the common operations that all route-finding strategies must provide. This allows different route algorithms to be used interchangeably through the same interface.

Trip: The *Context* class of the Strategy pattern. It holds a reference to a RouteStrategy object and delegates route-planning behaviour to whichever strategy is currently selected.

Concrete Route Strategies: These classes inherit from RouteStrategy and implement the behaviour defined by the interface. Each concrete strategy provides a different way of finding or planning a route.

2.3

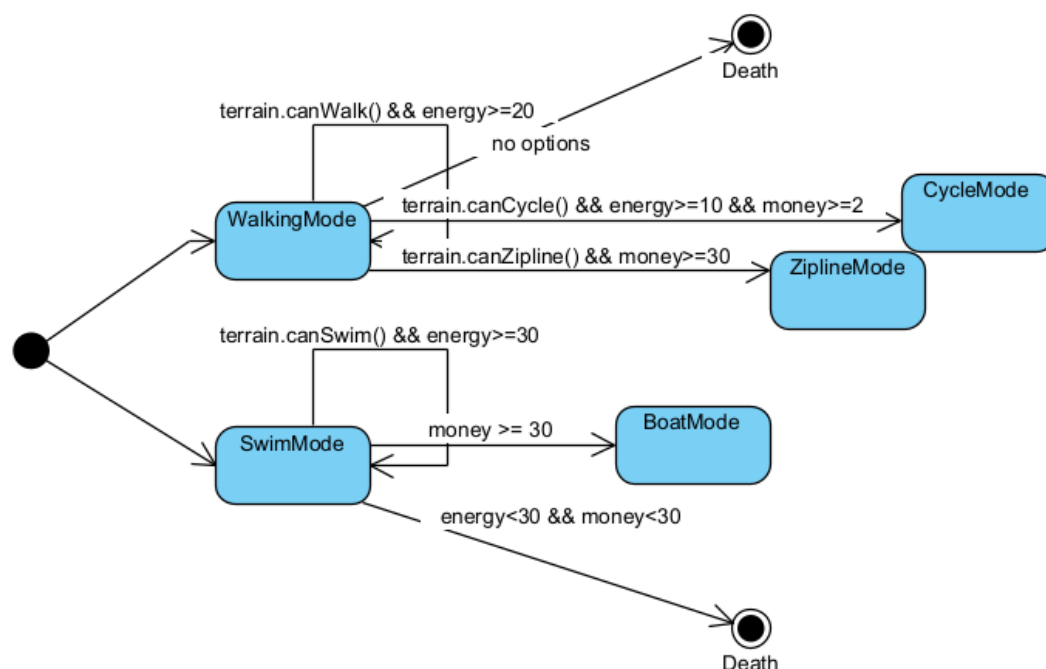
New biome: Dungeon, it required four classes to be made. A factory, terrain, NPC and obstacle. Which extends, BiomeFactory, Terrain, NPC, Obstacle respectively. It does not impact any existing code.

New Decorator: questBoard, which extends PlaceDecorator. Since it is only an added child, it doesn't impact any existing code and future decorators will be added easily.

New move: zipline, which extends TravelMode. It is a concrete state and so, doesn't impact any existing code because of the state design pattern. It just implements it's own behaviour by overriding move().

Task 3

3.3



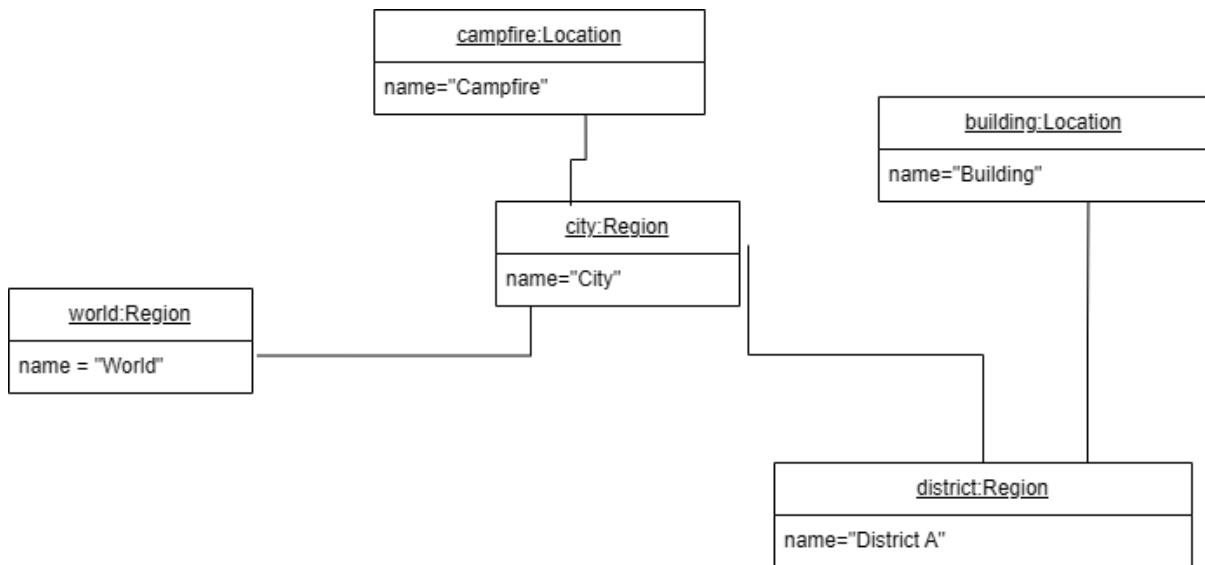
Task 4

4.3

Strategy and State may have the same structure, but they differ in intent. The Strategy pattern has different implementations that achieve the same result. There are different route strategies, but they can be interchanged. Swapping routes only changes the outcome of the next findRoute() call. Trip itself doesn't change what it fundamentally is. State (setMode) changes what it is, it changes the state. Strategy therefore cannot be interchanged with the State pattern because it does different things based on the state while the caller doesn't have to accommodate each state. TravelMode as Strategy would push the transition-menu logic out into the caller on every single move, treating RouteStrategy as State would have individual strategies

mutating Trip's state and referencing each other, scattering the stat-based selection rule across multiple classes instead of keeping it as one policy in Trip

Task 5.2



Task 6.2

